

LAB MANUAL



COM682

**Faculty of Computing, Engineering and
The Built Environment**

COM682 – Lab 03

Cosmos Database

Contents

1. What is Cosmos DB?	3
2. Prerequisites	3
3. Find the region where you can create a resource	4
4. Create a database account	5
Add a container	9
Add sample data.....	11
Query your data.....	13
Importing data in Azure Cosmos DB	15
Query Overview	17
Running your first query	17
Open Data Explorer	17
Dot and quoted property projection accessors	19
WHERE clauses	20
Advanced projection	20
ORDER BY clause	21
Limiting query result size	22
More advanced filtering	23
More advanced projection	23
.....	24
JOIN within your documents	24
System functions	25
Correlated sub-queries	26
Multi-value subqueries	26
Scalar sub-queries	27
Task 1:.....	28
Task 2:.....	28

1. What is Cosmos DB?

Azure Cosmos DB is a NoSQL Database. Azure Cosmos DB is Microsoft, a multi-model Database service "for managing data at planet-scale".

In this lab, you create and manage an Azure Cosmos DB SQL API account from the Azure portal and Visual Studio Code with a Python app cloned from GitHub. Azure Cosmos DB is a multi-model database service that lets you quickly create and query document, table, key-value, and graph databases with global distribution and horizontal scale capabilities.

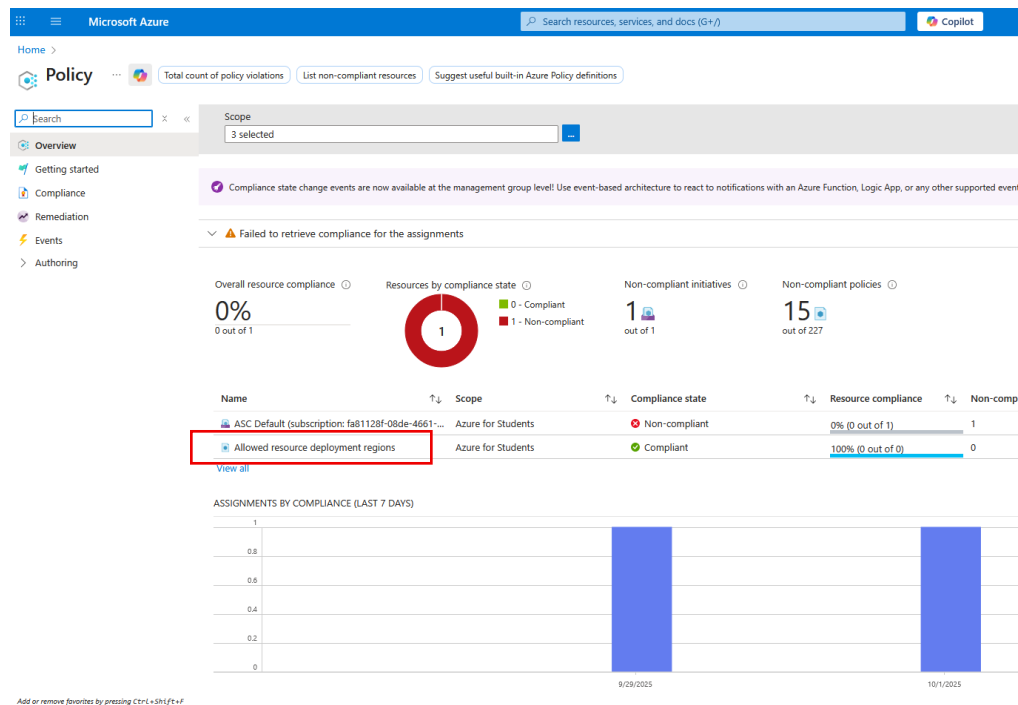
2. Prerequisites

- A Cosmos DB Account. You options are:
- Within an Azure active subscription:
- [Create an Azure free account](#) or use your existing subscription
- [Visual Studio Monthly Credits](#)
- [Azure Cosmos DB Free Tier](#)
- Without an Azure active subscription:
- [Try Azure Cosmos DB for free](#), a tests environment that lasts for 30 days.
- [Azure Cosmos DB Emulator](#)
- [Python 2.7 or 3.6+](#), with the python executable in your PATH.
- [Visual Studio Code](#).
- The [Python extension for Visual Studio Code](#).
- [Git](#).
- [Azure Cosmos DB SQL API SDK for Python](#)

3. Find the region where you can create a resource

Home - Microsoft Azure

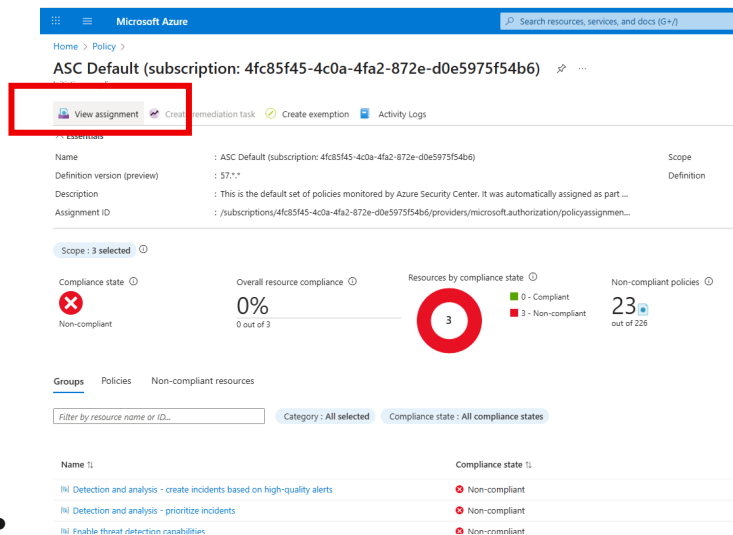
From the Azure portal menu or the **Home page**, search and select **Policy**.



The screenshot shows the Microsoft Azure Policy page. The left sidebar contains navigation links: Overview, Getting started, Compliance, Remediation, Events, and Authoring. The main content area displays a summary of policy violations and compliance. A red box highlights the 'Allowed resource deployment regions' link under the 'Non-compliant policies' section. Below this, there is a table of policy assignments and a bar chart showing compliance status over time.

Name	Scope	Compliance state	Resource compliance	Non-comp
ASC Default (subscription: fa81128f-08de-4661-...	Azure for Students	Non-compliant	0% (0 out of 1)	1
Allowed resource deployment regions	Azure for Students	Compliant	100% (0 out of 0)	0

- Click on 'Allowed resource Deployment regions'
 - Next page, click on View Assignment



The screenshot shows the Microsoft Azure Policy page for a specific assignment. The 'View assignment' link is highlighted with a red box. The page displays details about the assignment, including its name, scope, and compliance state. A table at the bottom lists the resources and their compliance status.

Name	Compliance state
[i] Detection and analysis - create incidents based on high-quality alerts	Non-compliant
[i] Detection and analysis - prioritize incidents	Non-compliant
[i] Enable threat detection capabilities	Non-compliant

- You can see list of allowed locations.

Assigned by : --		
Parameters (1) Resource selectors (0) Overrides (0) Exemptions (0) Remediation (0) Deployed resources Managed Identity		
Search by parameter name All types		
Parameter ID	Parameter name	Parameter value
listOfAllowedLocations	Allowed locations	["norwayeast","switzerlandnorth","uksouth","francecentral","polandcentral"]

*Note this location as you would need it for creating Cosmos DB.

4. Create a database account

- From the Azure portal menu or the **Home page**, select **Create a resource**.

[Home - Microsoft Azure](#)

On the **New** page, search for and select **Azure Cosmos DB**.

Home > Create a resource > Marketplace

Get Started

Service Providers

Management

Private Marketplace

Private Offer Management

My Marketplace

Favorites

My solutions

Recently created

Private plans

Categories

Machine Learning (24)

Analytics (19)

AI Apps and Agents (17)

Databases (14)

Storage (12)

IT & Management Tools (7)

Developer Tools (6)

Internet of Things (6)

Monitoring & Diagnostics

Search: azure cosmos db

Pricing: All Operating System: All Publisher Type: All

☐ Azure benefit eligible only ☐ Azure services only

New! Get AI-generated suggestions for 'azure cosmos db' [View suggestions](#)

Showing 1 to 20 of 64 results for 'azure cosmos db'. [Clear search](#)



Azure Cosmos DB

Microsoft

Azure Service

Globally-distributed, multi-model database service.

Create



Azure Cosmos DB Reserved Capacity

Microsoft

Azure Service

Reserved Instances (RIs) for Cosmos DB significantly reduce your Cosmos DB costs compared to pay-as-you-go prices.

Create



Azure Cosmos DB for MongoDB

Microsoft

Azure Service

Quickly setup a MongoDB compatible API database with built-in security and scalability.

Create



Azure Cosmos DB for MongoDB (vCore)

Microsoft

Azure Service

The best way to run your new and existing MongoDB workloads on Azure. Pay only for the compute/storage you use. Starts at Free

Create



DP-420 Workshop Designing and Deploying

Opssgility, LLC

SaaS

5-day workshop with OneVenue and Circuit Sandbox focused on Circuit.



Azure Databricks

Microsoft

Azure Service

Azure Databricks is the fast, easy and collaborative Apache Spark-based



Azure Managed Instance for Apache Cassandra

Microsoft

Azure Service

Azure Managed Instance for Apache Cassandra



Azure Database Migration Service

Microsoft

Azure Service

Azure Database Migration Service

2. On the **Azure Cosmos DB** page, select **Create**.
3. Select **Azure Cosmos DB for NoSQL**

[Home](#) > [Create a resource](#) > [Marketplace](#) >

Create an Azure Cosmos DB account ...

Which API best suits your workload?

Azure Cosmos DB is a fully managed NoSQL and relational database service for building scalable, high performance applications. [Learn more](#)

To start, select the API to create a new account. The API selection cannot be changed after account creation.

Recommended APIs Others

Azure Cosmos DB for NoSQL

Azure Cosmos DB's core, or native API for working with documents. Supports fast, flexible development with familiar SQL query language and client libraries for .NET, JavaScript, Python, and Java.

Create

[Learn more](#)

Azure Cosmos DB for MongoDB

Fully managed database service for apps written for MongoDB. Recommended if you have existing MongoDB workloads that you plan to migrate to Azure Cosmos DB.

Create

[Learn more](#)

Give Feedback

[Help improve this page](#)

4. In the **Create Azure Cosmos DB Account - Azure Cosmos DB for NoSQL** page, enter the basic settings for the new Azure Cosmos account.

Setting	Value	Description
Workload Type	Learning	Select the Workload Type Learning.
Subscription	Subscription name	Select the Azure subscription that you want to use for this Azure Cosmos account. (Azure for Students)
Resource Group	Resource group name	Select a resource group, or select Create new , then enter a unique name for the new resource group. <i>CosmosDBResource</i>
Account Name	A unique name	Enter a name to identify your Azure Cosmos account. Because <i>documents.azure.com</i> is appended to the name that you provide to create your URI, use a unique name.

		The name can only contain lowercase letters, numbers, and the hyphen (-) character. It must be between 3-44 characters in length. <i>For example : cosmosaccount123</i>
Availability Zones		Disable
Location	The region closest to your users	Select a geographic location to host your Azure Cosmos DB account. Use the location that is closest to your users to give them the fastest access to the data. Default = UK South (you know the location from earlier step by searching ‘policy’ as above)
Capacity mode	Serverless	Select Serverless to create an account in serverless mode.

Home > Create Azure Cosmos DB Account - Azure Cosmos DB for NoSQL

Basics Global distribution Networking Backup Policy Security Tags Review + create

Azure Cosmos DB is a fully managed NoSQL and relational database service for building scalable, high performance applications. Go to production starting at \$24/month per database, multiple containers included. [Learn more](#)

Project Details

Choose a workload type that best aligns with your goals. This helps us provide an optimized starting point for your Azure Cosmos DB account setup. Don't worry—no matter which workload type you choose, you can customize each setting to fit your needs or stick to the defaults provided.

Workload Type *

Best for beginners. Low cost, easy setup, and quick exploration to learn Azure Cosmos DB.

Select the subscription to manage deployed resources and costs. Use resource groups like folders to organize and manage all your resources.

Subscription *

Resource Group *

[Create new](#)

Instance Details

Account Name *

Configure availability zone settings for your account. You cannot change these settings once the account is created.

Availability Zones ☐ Enable ☒ Disable

Location *

Available locations are determined by your subscription's access and availability zone support (if that is enabled). If you don't see or cannot select your desired location, please open a support request for region access. [Click here for more details on how to create a region access request](#)

Unsure of your traffic pattern? Explore Azure Cosmos DB's serverless offering. If you're just starting out and skip capacity planning by only paying for the request units (RU) you consume. [Learn more](#)

Capacity mode ☐ Provisioned throughput ☒ Serverless

Request units per second (RU/s) are pre-configured and billed hourly, offering guaranteed throughput with two modes—**Autoscale**, which dynamically adjusts within a 1-10x range for varying workloads, and **Manual**, with fixed RU/s for steady, predictable traffic.

Serverless Ideal for intermittent or unpredictable traffic. Billed only for consumed RU/s, scaling on-demand without capacity planning or minimum charges. [Learn more about capacity mode](#)

5. In the Global Distribution tab, configure the following details. You can leave the default values for the purpose of this quickstart:

Setting	Value	Description
Geo-Redundancy	Disable	Enable or disable global distribution on your account by pairing your region with a pair region. You can add more regions to your account later.
Multi-region Writes	Disable (By Default)	Multi-region writes capability allows you to take advantage of the provisioned throughput for your databases and containers across the globe.

[Home](#) >

Create Azure Cosmos DB Account - Azure Cosmos DB for NoSQL

Basics Global distribution Networking Backup Policy Encryption Tags Review + create

Multiple regions are not supported with serverless capacity mode.

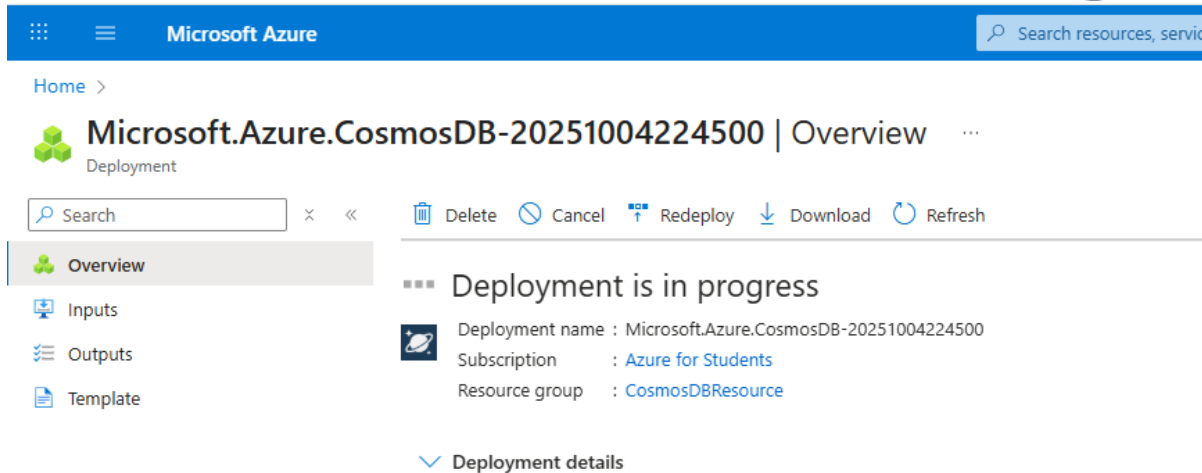
Geo-Redundancy ⓘ ☐ Enable ☒ Disable

Multi-region Writes ⓘ ☐ Enable ☒ Disable

6. Optionally you can configure additional details in the following tabs:

- **Networking** – Select **All Networks**
- **Backup Policy** - Configure either [periodic](#) or [continuous](#) backup policy. You can choose Periodic default selected for this lab.
- **Encryption** - Use either service-managed key or a [customer-managed key](#). You can choose default for this lab.
- **Tags** - Tags are name/value pairs that enable you to categorize resources and view consolidated billing by applying the same tag to multiple resources and resource groups.
- Select **Review + create**.

7. Review the account settings, and then select **Create**. It takes a few minutes to create the account. Wait for the portal page to display **Your deployment is complete**.

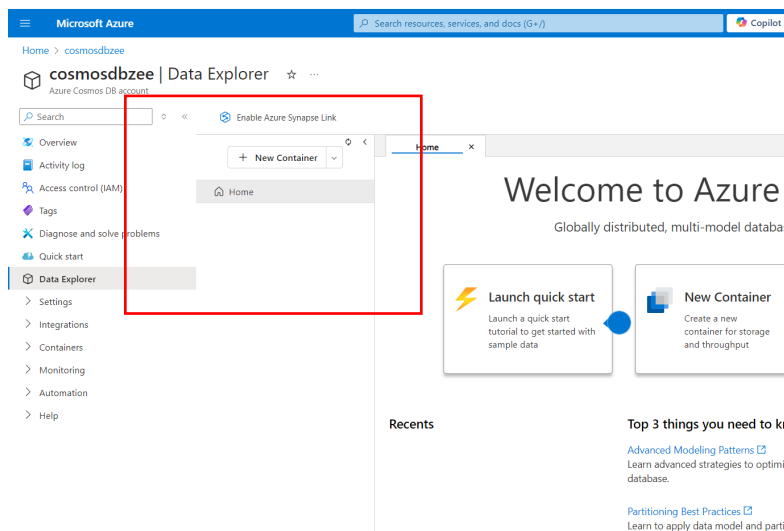


8. Once completed, Select **Go to resource** to go to the Azure Cosmos DB account page.

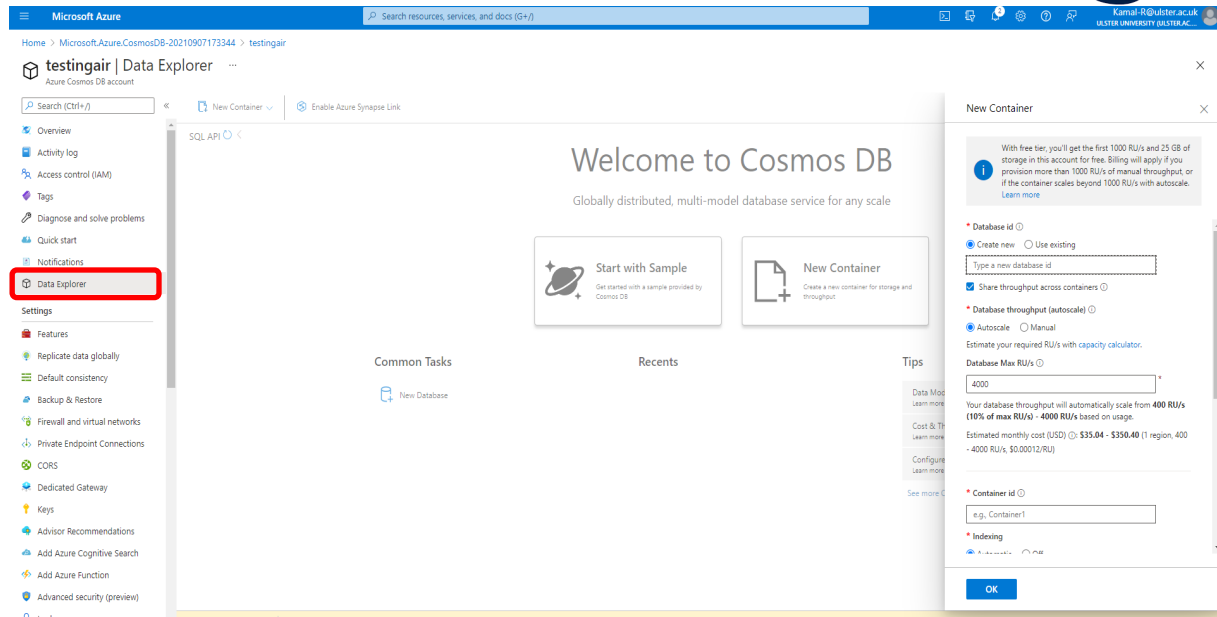
Add a container

You can now use the Data Explorer tool in the Azure portal to create a database and container.

1. Select **Data Explorer > New Container**.



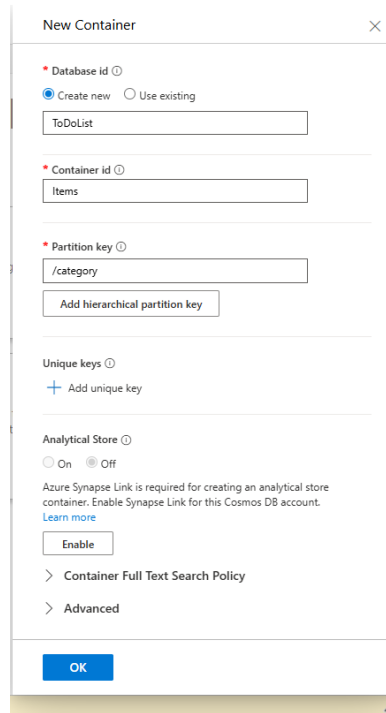
2. The **Add Container** area is displayed on the far right, or you can find it from Data explorer.



In the Add container page, enter the settings for the new container.

Setting	Suggested value	Description
Database ID	Create new: 'ToDoList'	Database names must contain from 1 through 255 characters, and they cannot contain /, \, #, ? or a trailing space.
Container ID	Items	Enter <i>Items</i> as the name for your new container. Container IDs have the same character requirements as database names.
Partition key	/category	The Sample described in this article uses <i>/category</i> as the partition key.

Don't add **Unique keys** or turn on the **Analytical store** for this example. Unique keys let you add a layer of data integrity to the Database by ensuring the uniqueness of one or more values per partition key. For more information, see Unique keys in Azure Cosmos DB. The analytical store is used to enable large-scale analytics against operational data without any impact on your transactional workloads.



New Container [X]

* Database id ⓘ
☒ Create new ☐ Use existing

* Container id ⓘ

* Partition key ⓘ

Unique keys ⓘ

Analytical Store ⓘ
☐ On ☒ Off
Azure Synapse Link is required for creating an analytical store container. Enable Synapse Link for this Cosmos DB account. [Learn more](#)

> Container Full Text Search Policy

> Advanced

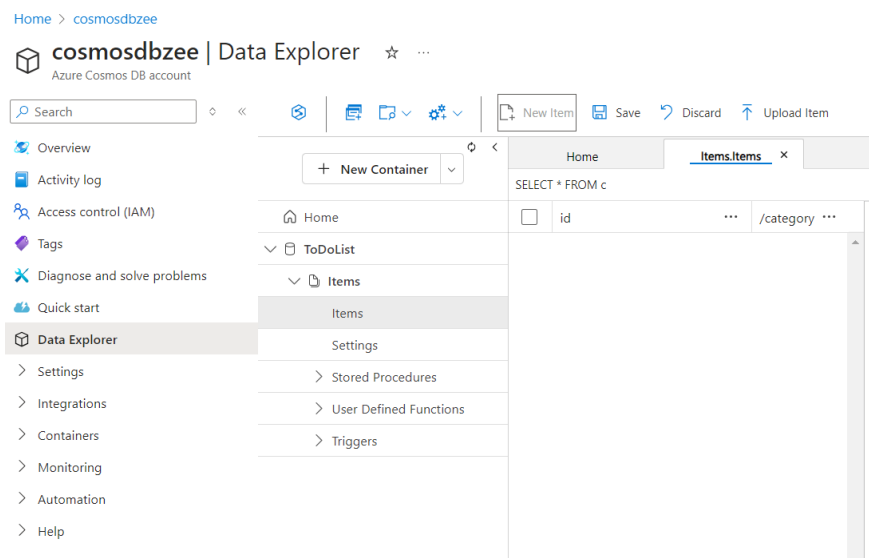
Select OK. The Data Explorer displays the new Database and container.

Add sample data

You can now add data to your new container using Data Explorer.

1. From the **Data Explorer**, expand the **ToDoList** database, expand the **Items** container.

Select **Items**, and then select **New Item**.



Home > cosmosdbzee

cosmosdbzee | Data Explorer ☆ ...
Azure Cosmos DB account

Search [] ◊ ◀ ▶ ⚙️

Home
 ▼ ToDoList
 ▼ Items
 Items
 Settings
 > Stored Procedures
 > User Defined Functions
 > Triggers

Home | **Items.Items** x

SELECT * FROM c

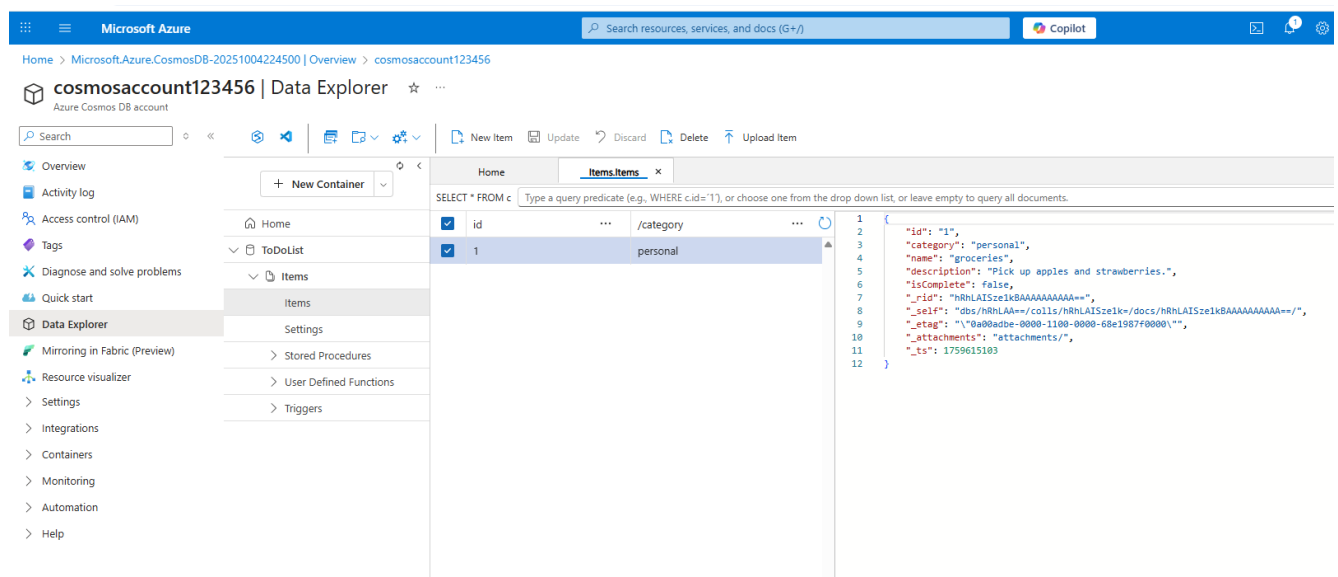
	id	...	/category	...
☐				

Overview
 Activity log
 Access control (IAM)
 Tags
 Diagnose and solve problems
 Quick start
Data Explorer
 > Settings
 > Integrations
 > Containers
 > Monitoring
 > Automation
 > Help

2. Now add a document to the container with the following structure.

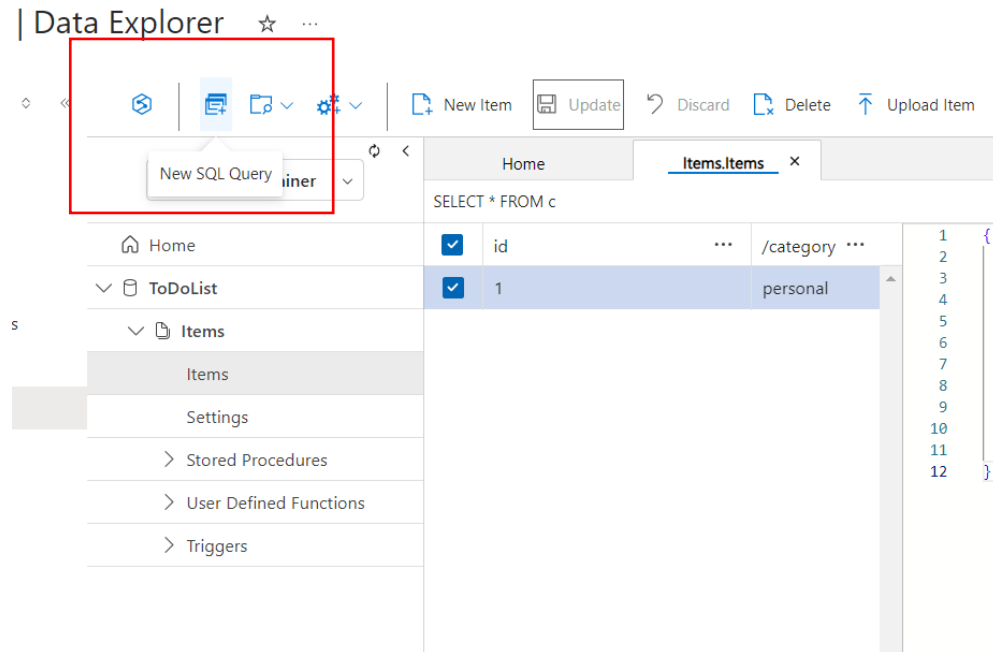
```
{
  "id": "1",
  "category": "personal",
  "name": "groceries",
  "description": "Pick up apples and strawberries.",
  "isComplete": false
}
```

3. Once you've added the json to the **Documents** tab, select **Save**.



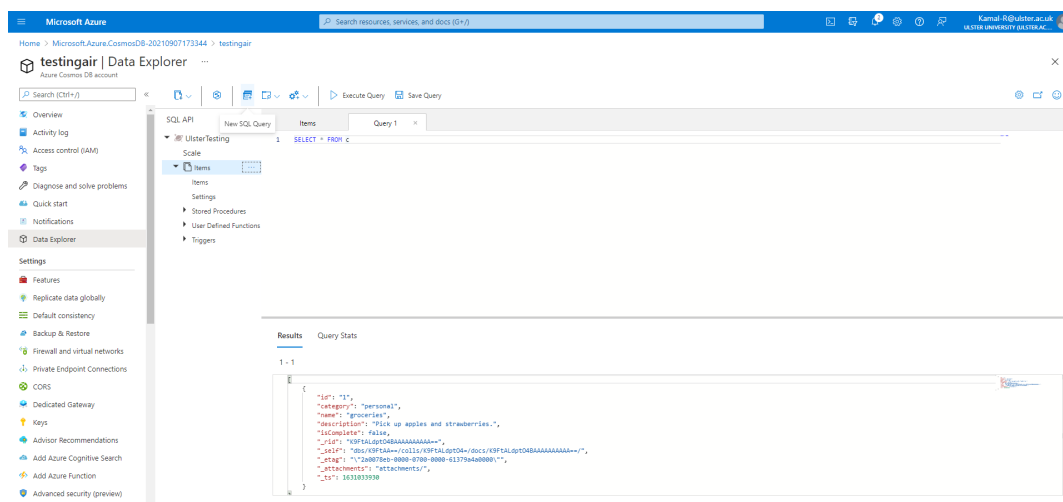
4. Create and save one more document where you insert a unique value for the id property, and change the other properties as you see fit. Your new documents can have any structure you want as Azure Cosmos DB doesn't impose any schema on your data.

Query your data



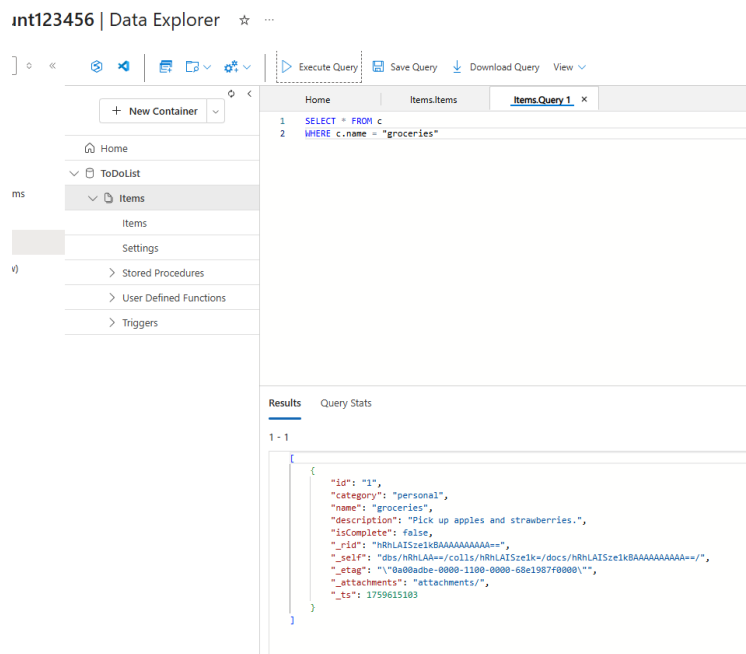
You can use queries in Data Explorer to retrieve and filter your data.

1. At the top of the **Items** tab in Data Explorer, review the default query `SELECT * FROM c`. This query retrieves and displays all documents from the container ordered by ID.



If you're familiar with SQL syntax, you can enter any supported [SQL queries](#) in the query predicate box. You can also use Data Explorer to create stored procedures, UDFs, and triggers for server-side business logic.

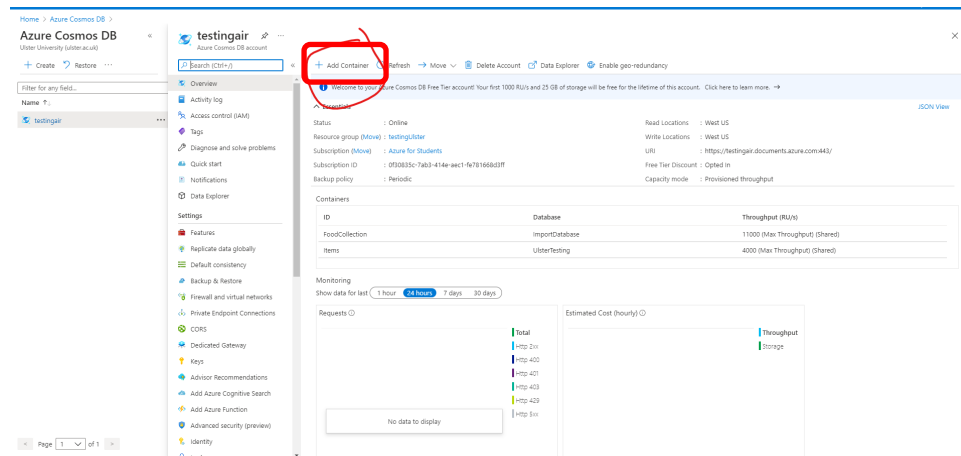
Same queries to extract c.name is below:



Data Explorer provides easy Azure portal access to all of the built-in programmatic data access features available in the APIs. You also use the portal to scale throughput, get keys and connection strings, and review your Azure Cosmos DB account metrics and SLAs.

Importing data in Azure Cosmos DB

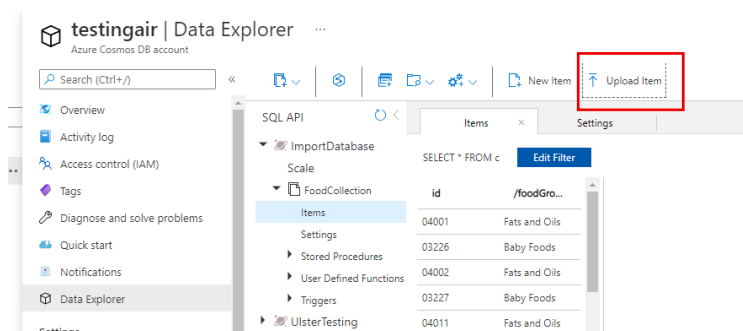
In this section we are going to upload the json file to Cosmos DB. Go to the your Azure Cosmos DB page.



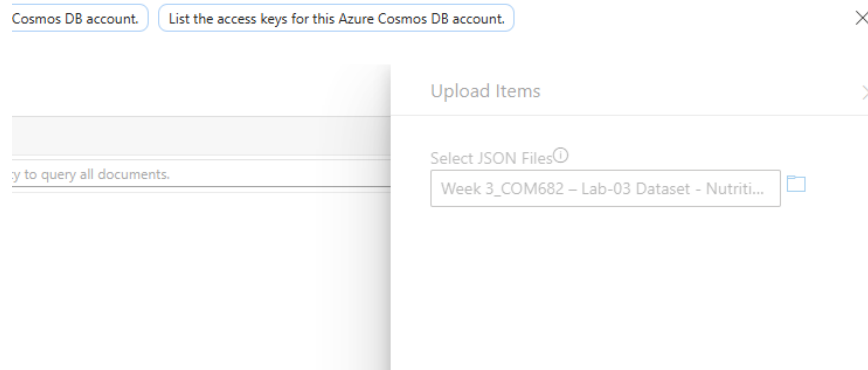
1. In the **Add Container** popup, perform the following actions:

1. In the **Database id** field, select the **Create new** option and enter the value **ImportDatabase**.
2. In the **Container Id** field, enter the value **FoodCollection**.
3. In the **Partition key** field, enter the value `/foodGroup`.
4. Select the **OK** button.

Please click on the **ImportDatabase** and click on items under Foodcollection.

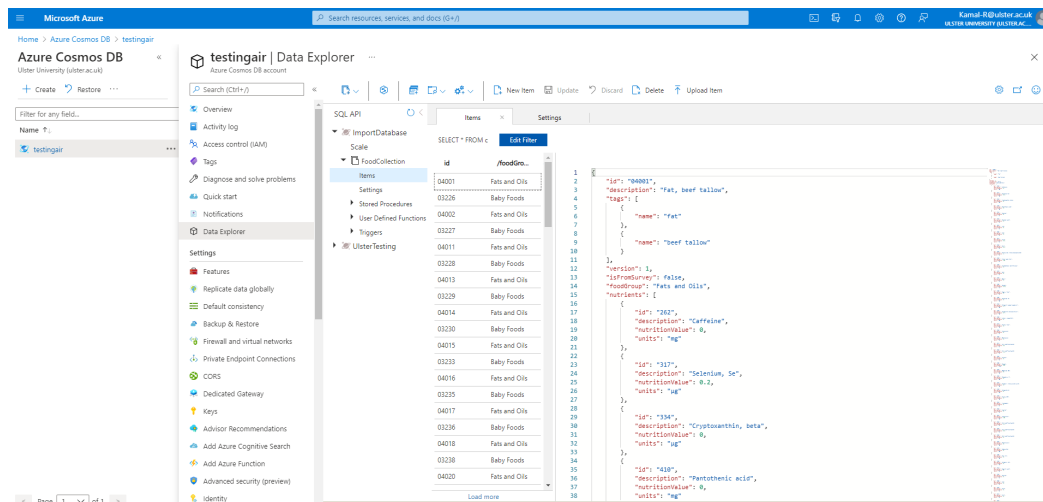


Click on upload item and upload the provided JSON file with this practical. You can find this file on BB with name 'Week-3_COM682_Lab-03 Dataset – NutritionData.json'



Please be patient the file will take time to upload at least 10 min or more.

After successful upload the data should be look like following snippet.



Azure Cosmos DB SQL API accounts provide support for querying items using the Structured Query Language (SQL), one of the most familiar and popular query languages, as a JSON query language. In this lab, you will explore how to use these rich query capabilities directly through the Azure Portal. No separate tools or client side code are required.

Query Overview

Querying JSON with SQL allows Azure Cosmos DB to combine the advantages of a legacy relational databases with a NoSQL database. You can use many rich query capabilities such as sub-queries or aggregation functions but still retain the many advantages of modeling data in a NoSQL database.

Azure Cosmos DB supports strict JSON items only. The type system and expressions are restricted to deal only with JSON types. For more information, see the [JSON specification](#).

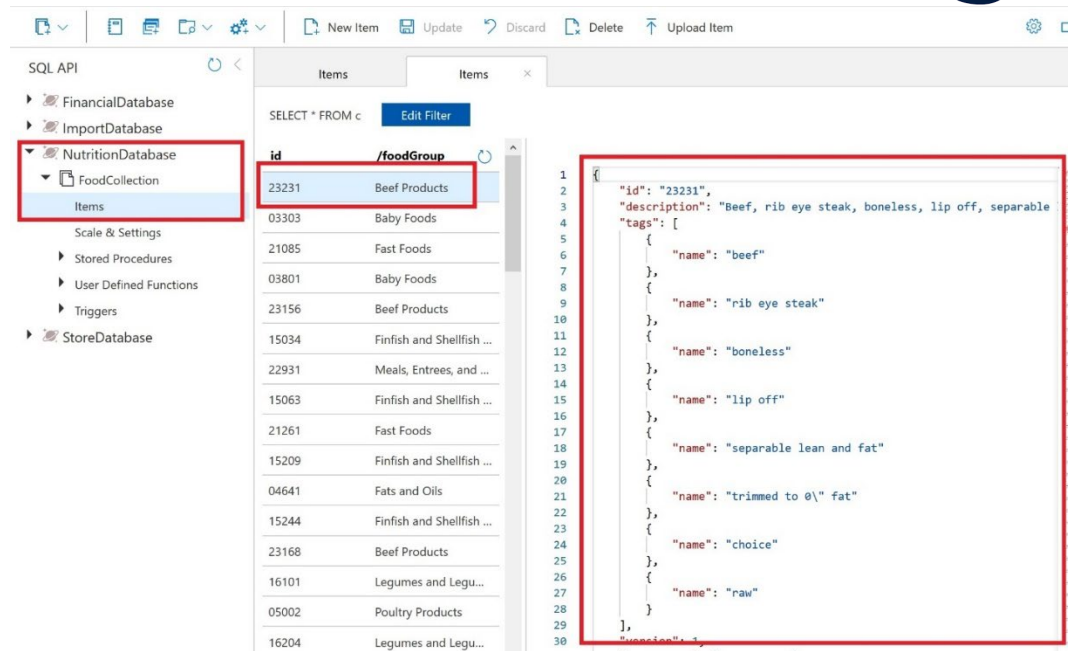
Running your first query

In this lab section, you will query your **FoodCollection**.

You will begin by running basic queries with `SELECT`, `WHERE`, and `FROM` clauses.

Open Data Explorer

1. Locate and select the **Data Explorer** link on the left side of the page.
2. In the **Data Explorer** section, expand the **NutritionDatabase** database node and then expand the **FoodCollection** container node.
3. Within the **FoodCollection** node, select the **Items** link.
4. View the items within the container. Observe how these documents have many properties, including arrays.



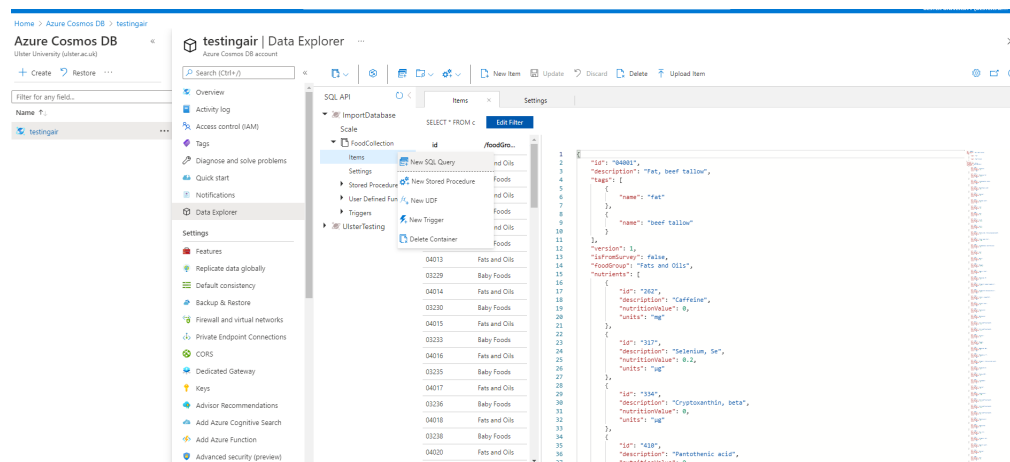
The screenshot shows the SQL API interface. On the left, the 'NutritionDatabase' is expanded, showing 'FoodCollection' and 'Items'. The 'Items' table is selected, and the 'id' and 'foodGroup' columns are highlighted. The table contains the following data:

id	foodGroup
23231	Beef Products
03303	Baby Foods
21085	Fast Foods
03801	Baby Foods
23156	Beef Products
15034	Finfish and Shellfish ...
22931	Meals, Entrees, and ...
15063	Finfish and Shellfish ...
21261	Fast Foods
15209	Finfish and Shellfish ...
04641	Fats and Oils
15244	Finfish and Shellfish ...
23168	Beef Products
16101	Legumes and Legu...
05002	Poultry Products
16204	Legumes and Legu...

On the right, a JSON response for item 23231 is shown:

```
{
  "id": "23231",
  "description": "Beef, rib eye steak, boneless, lip off, separable",
  "tags": [
    {
      "name": "beef"
    },
    {
      "name": "rib eye steak"
    },
    {
      "name": "boneless"
    },
    {
      "name": "lip off"
    },
    {
      "name": "separable lean and fat"
    },
    {
      "name": "trimmed to 0\ fat"
    },
    {
      "name": "choice"
    },
    {
      "name": "raw"
    }
  ]
}
```

5. Select New SQL Query.



The screenshot shows the Azure Cosmos DB Data Explorer interface. On the left, the 'testngair' database is expanded, showing 'FoodCollection' and 'Items'. The 'Items' table is selected, and the 'id' and 'foodGroup' columns are highlighted. The table contains the following data:

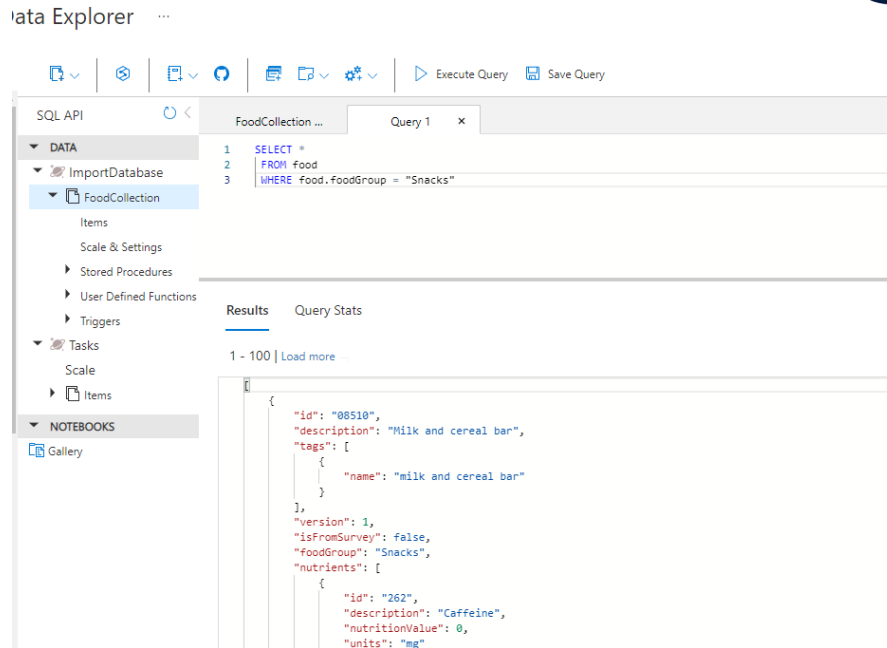
id	foodGroup
40013	Fats and Oils
03229	Baby Foods
04014	Fats and Oils
03230	Baby Foods
04015	Fats and Oils
03233	Baby Foods
04016	Fats and Oils
03235	Baby Foods
04017	Fats and Oils
03236	Baby Foods
04018	Fats and Oils
03238	Baby Foods
04020	Fats and Oils

On the right, a JSON response for item 40001 is shown:

```
{
  "id": "40001",
  "description": "Fat, beef tallow",
  "tags": [
    {
      "name": "fat"
    },
    {
      "name": "beef tallow"
    }
  ],
  "version": 1,
  "isReserved": false,
  "foodGroup": "Fats and Oils",
  "nutrition": {
    "fat": "36g",
    "description": "Caffeine",
    "nutritionValue": 0,
    "units": "mg"
  },
  "id": "40017",
  "description": "Selenium, Se",
  "nutritionValue": 0.2,
  "units": "ug"
},
{
  "id": "40018",
  "description": "Cryptoxanthin, beta",
  "nutritionValue": 0,
  "units": "ug"
},
{
  "id": "40019",
  "description": "Pantothenic acid",
  "nutritionValue": 0,
  "units": "mg"
}
```

6. Paste the following SQL query and select **Execute Query**.

```
SELECT *
FROM food
WHERE food.foodGroup = "Snacks"
```



- You will see that the query returned the single document foodGroup is “Snacks”. Explore the structure of this item as it is representative of the items within the **FoodCollection** container that we will be working with for the remainder of this section.

Dot and quoted property projection accessors

You can choose which properties of the document to project into the result using the dot notation.

Select **New SQL Query**. Paste the following SQL query and selecting **Execute Query**.

```
SELECT food.id
FROM food
WHERE food.foodGroup = "Snacks"
```

Though less common, you can also access properties using the quoted property operator [""]. For example, SELECT food.id and SELECT food["id"] are equivalent. This syntax is useful to escape a property that contains spaces, special characters, or has the same name as a SQL keyword or reserved word.

```
SELECT food["id"]  
FROM food  
WHERE food["foodGroup"] = "Snacks"
```

WHERE clauses

Let's explore WHERE clauses. You can add complex scalar expressions including arithmetic, comparison and logical operators in the WHERE clause.

Run the below query by selecting the **New SQL Query**.

Paste the following SQL query and then select **Execute Query**.

```
SELECT food.id,  
food.description,  
food.tags,  
food.foodGroup,  
food.version  
FROM food  
WHERE (food.foodGroup = "Baby Foods")
```

This query will return the id, description, servings, tags, foodGroup, manufacturerName and version for items with “Baby Foods” for **foodGroup** and a **version** greater than 0.

You should note that where the query returned the results of tags node it projected the entire contents of the property which in this case is an array.

Advanced projection

Azure Cosmos DB supports several forms of transformation on the resultant JSON. One of the simplest is to alias your JSON elements using the AS aliasing keyword as you project your results.

By running the query below you will see that the element names are transformed. In addition, the projection is accessing only the first element in the servings array for all items specified by the WHERE clause.

Run the below query by selecting the **New SQL Query**.

Paste the following SQL query and then select **Execute Query**.

```
SELECT food.description,  
food.foodGroup,  
food.servings[0].description AS servingDescription,  
food.servings[0].weightInGrams AS servingWeight  
FROM food  
WHERE food.foodGroup = "Fruits and Fruit Juices"  
AND food.servings[0].description = "cup"
```

ORDER BY clause

Azure Cosmos DB supports adding an ORDER BY clause to sort results based on one or more properties

Run the below query by selecting the **New SQL Query**.

Paste the following SQL query and then select **Execute Query**.

```
SELECT food.description,  
food.foodGroup,  
food.servings[0].description AS servingDescription,  
food.servings[0].weightInGrams AS servingWeight  
FROM food  
WHERE food.foodGroup = "Fruits and Fruit Juices" AND food.servings[0].description = "cup"  
ORDER BY food.servings[0].weightInGrams DESC
```

You can learn more about configuring the required indexes for an Order By clause in the later Indexing Lab or by reading [our docs](#).

Limiting query result size

Azure Cosmos DB supports the TOP keyword. TOP can be used to limit the number of returning values from a query.

Run the query below to see the top 20 results.

```
SELECT TOP 20 food.id,  
food.description,  
food.tags,  
food.foodGroup  
FROM food  
WHERE food.foodGroup = "Snacks"
```

The OFFSET LIMIT clause is an optional clause to skip then take some number of values from the query. The OFFSET count and the LIMIT count are required in the OFFSET LIMIT clause.

```
SELECT food.id,  
food.description,  
food.tags,  
food.foodGroup  
FROM food  
WHERE food.foodGroup = "Snacks"  
ORDER BY food.id  
OFFSET 10 LIMIT 10
```

When OFFSET LIMIT is used in conjunction with an ORDER BY clause, the result set is produced by doing skip and take on the ordered values. If no ORDER BY clause is used, it will result in a deterministic order of values.



More advanced filtering

Let's add the IN and BETWEEN keywords into our queries. IN can be used to check whether a specified value matches any element in a given list and BETWEEN can be used to run queries against a range of values.

Run the query below:

```
SELECT food.id,  
       food.description,  
       food.tags,  
       food.foodGroup,  
       food.version  
FROM food  
WHERE food.foodGroup IN ("Poultry Products", "Sausages and Luncheon Meats")  
      AND (food.id BETWEEN "05740" AND "07050")
```

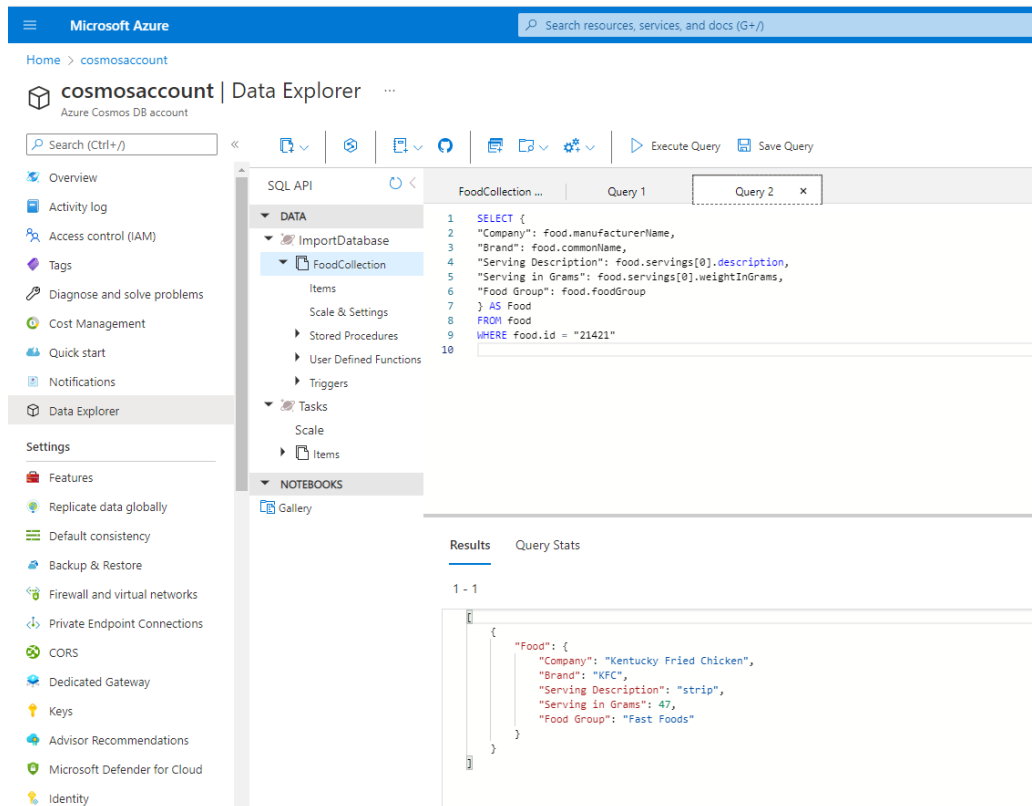
More advanced projection

Azure Cosmos DB supports JSON projection within its queries. Let's project a new JSON Object with modified property names.

Run the query below to see the results.

```
SELECT {  
  "Company": food.manufacturerName,  
  "Brand": food.commonName,  
  "Serving Description": food.servings[0].description,  
  "Serving in Grams": food.servings[0].weightInGrams,  
  "Food Group": food.foodGroup  
} AS Food  
FROM food
```

WHERE food.id = "21421"



The screenshot shows the Microsoft Azure Data Explorer interface. The left sidebar contains navigation options like Overview, Activity log, Access control (IAM), Tags, Diagnose and solve problems, Cost Management, Quick start, Notifications, Data Explorer, and Settings. The main pane displays the 'FoodCollection' database structure. A query is executed in the 'Query 1' tab, showing the following SQL code:

```
1 SELECT {
2   "Company": food.manufacturerName,
3   "Brand": food.commonName,
4   "Serving Description": food.servings[0].description,
5   "Serving in Grams": food.servings[0].weightInGrams,
6   "Food Group": food.foodGroup
7 } AS Food
8 FROM food
9 WHERE food.id = "21421"
10
```

The results pane shows a single result (1 - 1) with the following JSON output:

```
{
  "Food": {
    "Company": "Kentucky Fried Chicken",
    "Brand": "KFC",
    "Serving Description": "strip",
    "Serving in Grams": 47,
    "Food Group": "Fast Foods"
  }
}
```

JOIN within your documents

Azure Cosmos DB's JOIN supports intra-document and self-joins. Azure Cosmos DB does not support JOINS across documents or containers.

In an earlier query example we returned a result with attributes of just the first serving of the food.servings array. By using the join syntax below, we can now return an item in the result for every item within the serving array while still being able to project the attributes from elsewhere in the item.

Run the query below to iterate on the food document's servings.

```
SELECT
food.id as FoodID,
serving.description as ServingDescription
FROM food
JOIN serving IN food.servings
WHERE food.id = "03226"
```

JOINS are useful if you need to filter on properties within an array. Run the below example that has filter after the intra-document JOIN.

```
SELECT VALUE COUNT(1)
FROM c
JOIN t IN c.tags
JOIN s IN c.servings
WHERE t.name = 'infant formula' AND s.amount > 1
```

System functions

Azure Cosmos DB supports a number of built-in functions for common operations. They cover mathematical functions like ABS, FLOOR and ROUND and type checking functions like IS_ARRAY, IS_BOOL and IS_DEFINED. [Learn more about supported system functions.](#)

Run the query below to see example use of some system functions

```
SELECT food.id,
food.commonName,
food.foodGroup,
ROUND(nutrient.nutritionValue) AS amount,
nutrient.units
FROM food JOIN nutrient IN food.nutrients
```

```
WHERE IS_DEFINED(food.commonName)
AND nutrient.description = "Water"
AND food.foodGroup IN ("Sausages and Luncheon Meats", "Legumes and Legume Products")
```

Correlated sub-queries

In many scenarios, a sub-query may be effective. A correlated sub-query is a query that references values from an outer query. We will walk through some of the most useful examples here. You can [learn more about subqueries](#).

There are two types of sub-queries: Multi-value sub-queries and scalar sub-queries. Multi-value sub-queries return a set of documents and are always used within the FROM clause. A scalar sub-query expression is a sub-query that evaluates to a single value.

Multi-value subqueries

You can optimize JOIN expressions with a sub-query.

Consider the following query which performs a self-join and then applies a filter on name, nutritionValue, and amount. We can use a sub-query to filter out the joined array items before joining with the next expression.

```
SELECT VALUE COUNT(1)
FROM c
JOIN t IN c.tags
JOIN n IN c.nutrients
JOIN s IN c.servings
WHERE t.name = 'infant formula' AND (n.nutritionValue > 0
AND n.nutritionValue < 10) AND s.amount > 1
```

We could rewrite this query using three sub-queries to optimize and reduce the Request Unit (RU) charge. Observe that the multi-value sub-query always appears in the FROM clause of the outer query.

```
SELECT VALUE COUNT(1)
FROM c
JOIN (SELECT VALUE t FROM t IN c.tags WHERE t.name = 'infant formula')
JOIN (SELECT VALUE n FROM n IN c.nutrients WHERE n.nutritionValue > 0 AND
n.nutritionValue < 10)
JOIN (SELECT VALUE s FROM s IN c.servings WHERE s.amount > 1)
```

Scalar sub-queries

One use case of scalar sub-queries is rewriting ARRAY_CONTAINS as EXISTS.

Consider the following query that uses ARRAY_CONTAINS:

```
SELECT TOP 5 f.id, f.tags
FROM food f
WHERE ARRAY_CONTAINS(f.tags, {name: 'orange'})
```

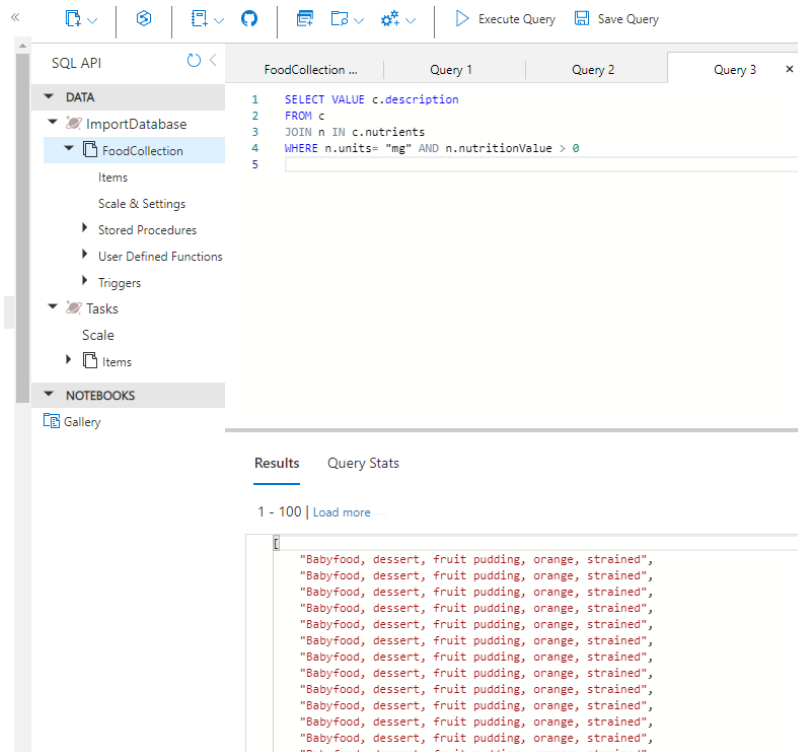
Run the following query which has the same results but uses EXISTS:

```
SELECT TOP 5 f.id, f.tags
FROM food f
WHERE EXISTS(SELECT VALUE t FROM t IN f.tags WHERE t.name = 'orange')
```

The major advantage of using EXISTS is the ability to have complex filters in the EXISTS function, rather than just the simple equality filters which ARRAY_CONTAINS permits.

Here is an example:

```
SELECT VALUE c.description
FROM c
JOIN n IN c.nutrients
WHERE n.units= "mg" AND n.nutritionValue > 0
```



Task 1:

Write in your own words the difference between SQL and NoSQL?

Task 2:

Read the suggested documentation for CosmosDB with the goal of understanding a) its APIs and b) how to integrate it into the web development language of your choice. This understanding will provide a good foundation for developing PaaS applications.

General Cosmos Documentation:

<https://docs.microsoft.com/en-us/azure/cosmos-db/>

Azure Learn - Choose the appropriate API for Azure Cosmos DB:

<https://docs.microsoft.com/en-us/learn/modules/choose-api-for-cosmos-db/1-introduction>

Azure Learn - Introduction to Azure Cosmos DB SQL API:

<https://docs.microsoft.com/en-us/learn/modules/intro-to-azure-cosmos-db-core-api/>



NOTE: If Free tier or Serverless is not selected then large costs may be incurred.

Serverless consumes credit in a miniscule manner. Alternatively you may configure this within the free tier limitations (recommended).

At the time of writing this was 400RU/s provisioned throughput with 25GB of storage. Check your free tier allowance.